WRENFORD BOROUGH COUNCIL · STREET CARE OFFICE

StreetCare AI Architecture & Workflow Guide

End to end: how the system is built, how every flow works, and the best practice for building and running it.

Part 1 · Overview

1 What the system does
2 Design principles
3 System architecture
4 Technology stack
5 Project structure
6 Configuration

Part 2 · End-to-end workflow

7 The whole journey
8 Database setup
9 Data pipeline
10 Model training
11 Knowledge indexing
12 Service start-up
13 Report queue
14 Report screening
15 Reading documents
16 Predictions
17 Review workflow
18 AI agents
19 Human decision gate
20 Decision documents
21 API and access control
22 Chat assistant
23 Dashboard
24 Storage and audit

Part 3 · Engineering practice

25 Build order
26 Testing strategy
27 Error handling
28 Security
29 Observability
30 Performance
31 Running and operating
32 Definition of done
33 Glossary

Colour key used in every diagram

Input data · Code step · ML model · AI agent (Gemini) · Person / refusal · Output / storage · Screen

> **Status of the code.** The repository holds `config/config.py`, the data and the PDFs. Every other module is empty. This guide describes the system as the specification (`StreetCare_AI.pdf`) defines it, and how to build it well. Function, file, table and field names are the exact ones the specification requires.

PART 1

Overview

What the system is for, the rules it never breaks, and how it is put together.

1 What the system does

Wrenford Borough Council looks after **48 streets** in **4 districts** (Town Centre, North, South, Villages). Residents, businesses, councillors, bus operators and schools report street defects through **3 channels** (website, StreetCare app, contact centre). Each report is one of **5 defect types**, each with its own written standard:

Defect type	Examples	Part of the street
Carriageway	pothole, sunken or broken road surface	Carriageway
Footway	broken or rocking paving slab, trip	Footway
Drainage	blocked gully, standing water	Carriageway
Vegetation	tree or hedge hanging too low	neither
Street lighting	lamp out, burning by day, leaning column	Footway

The three problems it solves

1 · Every channel writes differently
Report numbers in 3 formats; depths in mm, cm or inches; heights in m or feet; hours or minutes. An unconverted reading meets or misses the wrong limit.

2 · Standards look alike
The carriageway and footway standards word their depth rule the same way but set different limits. Reading the wrong one passes or fails the wrong reports.

3 · Documents disagree
A note attached to a report may give a different measurement from the report ("2.1 m clear" against 2.4). A busy coordinator can miss it.

+ · The street itself
Live street works, a traffic-sensitive street or a tall lighting column mean the repair must be coordinated first.

What the system produces

For every open report: a screening against its own standard (and its linked type's), the street works on the street, the coordination triggers, a reading of every attached sentence, a repeat likelihood, expected crew hours, and a **recommended decision**, and then it **stops**. A Highways Inspector decides. Only then does the system write a works order and resident update, an inspection request, or a coordination brief.

Nothing is repaired on the word of a model. No repair is ordered, written or sent until a human decides.

2 Design principles

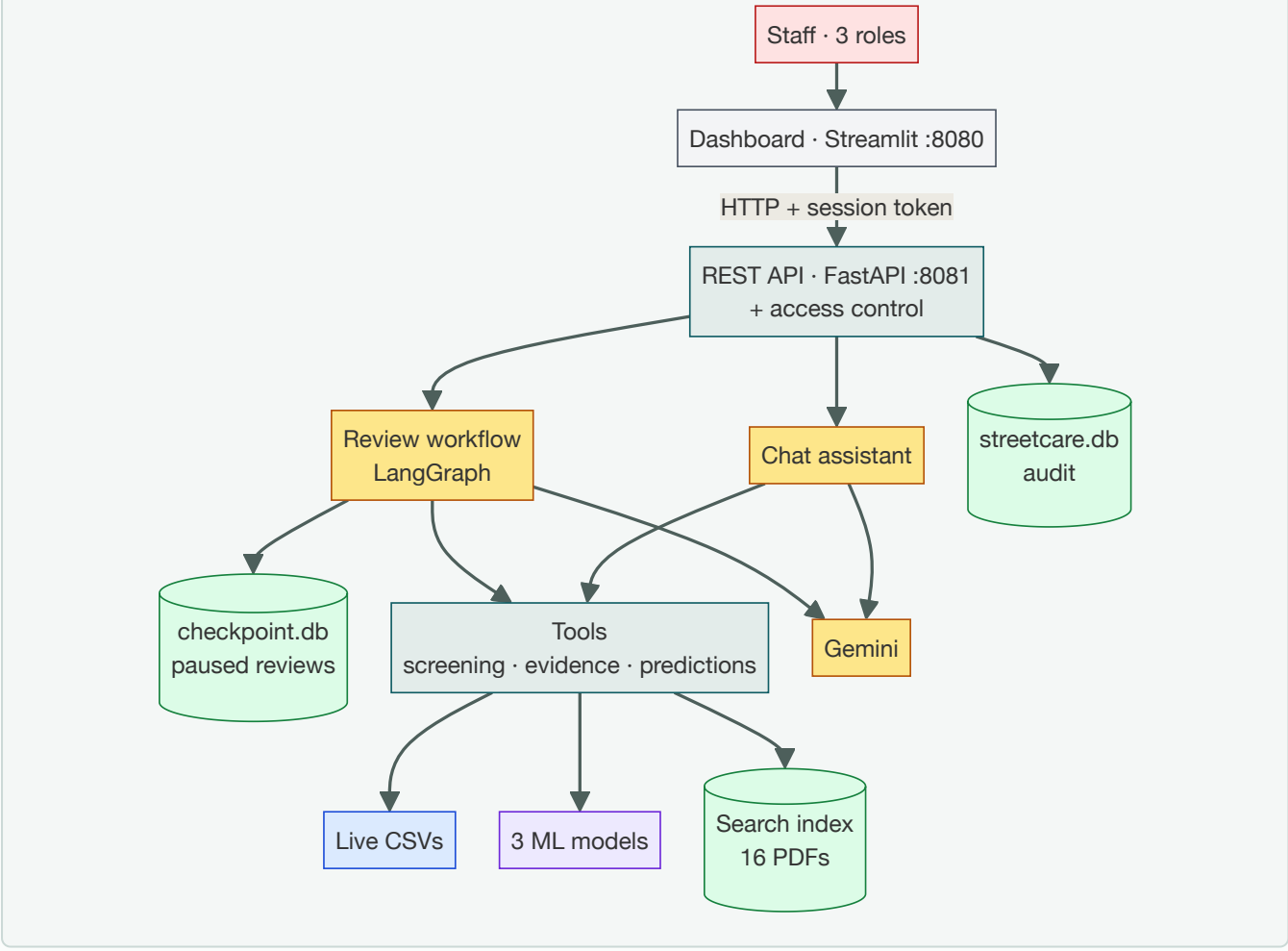
These rules shape every module. When a design choice is unclear, go back to them.

Principle	What it means in practice
Rules live in documents	No requirement and no limit is written in code or config. Every figure is read from the defect standard PDF at run time.
Never guess	An unknown spelling, unit or identifier stops the run with an error that names the column and the values.
Tools calculate, AI explains	Every number comes from deterministic code or an ML model. Each agent is one LLM call that only interprets facts it is given.
A person decides	The workflow pauses before any decision. Only a Highways Inspector appointed for the district, who did not open the review, can decide.
Predictions inform, never decide	Likelihood and crew hours never order the queue, never set the assessment and never appear in documents sent out.
Exact arithmetic	Conversions use `Decimal`, rounded half up once, at the reading's own precision.
Everything is audited	Every run, review, decision, accepted LLM call (tokens and time) and chat question is stored.
Privacy by default	Customer services never hear a prediction; no street figure is given from fewer than 5 reports.
Survive restarts	A review waiting for a decision is saved to disk and can be resumed by another process.
Exact names	Function, parameter, key, table and column names follow the specification exactly; a marking suite reads them.

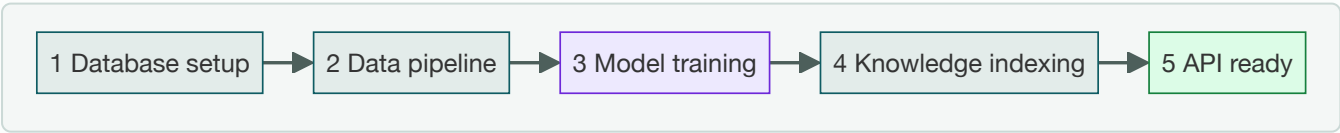
3 System architecture

The system has an **offline build**, run once in order, and an **online service** that staff use every day.

Online service (used every day)



Offline build (run once, in order)



Layers

Layer	Folder	Job	Depends on
Configuration	`config/`	Paths, mappings, patterns, thresholds, roles, messages	nothing
Data preparation	`data_pipeline/`	Read, clean, convert, profile	config
Modelling	`modeling/`	Train, score, save, load models	data_pipeline
Knowledge	`knowledge/`	PDF text, sections, index, standards	config

Tools	`tools/`	All deterministic facts about a report	data_pipeline, knowledge, modeling
Persistence	`persistence/`	Tables and repository	config
Agentic	`agentic/`	Agents, nodes, graph, checkpoint	tools, persistence
Chat	`chat/`	Document answers, record answers	knowledge, persistence
Access + API	`access/`, `api/`	Sign-in, permissions, routes	agentic, chat, persistence
Dashboard	`dashboard/`	Screens only	API over HTTP only

BEST PRACTICE

- Dependencies point one way: down the table. A lower layer never imports a higher one.
- The dashboard never imports project modules; it only calls the API.

4 Technology stack

Concern	Choice	Why
Data	pandas 3, Decimal	Tabular cleaning; exact unit conversion
ML	scikit-learn, joblib	Logistic regression, random forest, pipelines; compressed model files
PDF	pypdf	Text extraction from the 16 documents
Vector search	Chroma + sentence-transformers `all-MiniLM-L6-v2` (CPU, cosine)	Local, file-based index; traceable passages
Workflow	LangGraph + SQLite checkpointer	State machine with a human pause that survives restarts
LLM	Google Gemini (`google-genai`), temperature 0.2, max 8192 tokens	Interpretation and writing only
Database	SQLite via SQLAlchemy	Simple, auditable, single file
API	FastAPI, Pydantic, Uvicorn	Validation, auto docs at `/docs`
UI	Streamlit	Role-based pages over the API
Tests	pytest	Unit and integration tests

Ports and files

Item	Value
API	`0.0.0.0:8081`, docs at `http://localhost:8081/docs`
Dashboard	`0.0.0.0:8080`
Main database	`database/streetcare.db`
Workflow checkpoint	`database/checkpoint.db`
Vector index	`database/chroma_store`, collection `street_care_documents`
Secrets	`.env`: `GEMINI_API_KEY`, `GEMINI_MODEL`

5 Project structure

```
Project/
├── config/config.py              all settings (supplied complete)
├── data/
│   ├── report_history_{train,eval}.csv    repeat-model training data
│   ├── repair_logs_{train,eval}.csv       crew-hours-model training data
│   ├── statement_sentences.csv            sentence-model training data
│   ├── input/                             live: open reports, documents, street works register
│   └── docs/                              16 PDFs: 5 standards, 8 guides, policy, rules, casebook
├── data_pipeline/   dataset_loader · common_cleaning · report/history/repair_log/sentence/
│                    document/register_cleaning · feature_builder · quality_profiler
├── modeling/        model_registry · model_evaluation · repeat_model · crew_hours_model ·
sentence_model
├── knowledge/       pdf_extraction · chunk_splitting · vector_index · standard_reading
├── tools/           report_data_tools · screening_tools · evidence_tools · prediction_tools ·
│                    retrieval_tools · decision_tools · review_document_tools
├── persistence/     database · models (6 tables) · repository
├── agentic/         state · language_model · agents (7) · nodes (10) · graph
├── chat/            conversation_memory · street_care_assistant · record_answer
├── access/          password_security · permissions
├── api/             schemas · routes
├── dashboard/       api_client · sign_in · report_queue · report_detail · decision_queue ·
│                    audit_trail · street_care_chat
├── run_database_setup.py      1
├── run_data_pipeline.py       2
├── run_model_training.py      3
├── run_knowledge_indexing.py  4
├── run_api_service.py         5
├── run_dashboard.py           6
└── tests.py

Created at run time:  processed/   output/   output/models/   database/
```

6 Configuration

`config/config.py` is supplied complete and is the single source of every setting **except** the requirements and their limits, which are read from the PDFs.

Group	Examples
Paths	`RAW_*_FILE`, `CLEAN_*_FILE`, `*_MODEL_FILE`, `STREETCARE_DATABASE_FILE`
Missing values	`NULL_TOKENS` = "", n/a, na, -, null, not known, caller unsure
Spelling mappings	`DEFECT_TYPE_MAPPING`, `STREET_NAME_MAPPING`, `FORM_VALUE_MAPPINGS` …
Units	`MILLIMETRES_PER_INCH` 25.4, `METRES_PER_FOOT` 0.3048, `FORM_READINGS`
Patterns	`REPORT_REFERENCE_PATTERN`, `SECTION_HEADING_PATTERN`, `REQUIREMENT_SENTENCE_PATTERNS`
Decisions	`DECISION_ORDER/INSPECT/REFER`, `DECISIONS_OPEN`, `LLM_CALLS_BY_DECISION`
Thresholds	likelihood bands 0.20 / 0.40, overrun ×1.3, high column 8 m, group size 5, model minimums
Roles	`ROLE_PERMISSIONS`, `ROLE_PAGE_ACCESS`, `PERMISSION_BY_ROUTE`, `SEED_USER_ACCOUNTS` (16)
Messages	every refusal and template the system prints

BEST PRACTICE

- Always `import config` and use the constant. Never retype a string such as `"ORDER REPAIR"` or a path.
- Do not edit `config.py`; it is supplied and read by the marking suite.
- Run every command from the `Project/` folder: all paths are relative to it.

AVOID

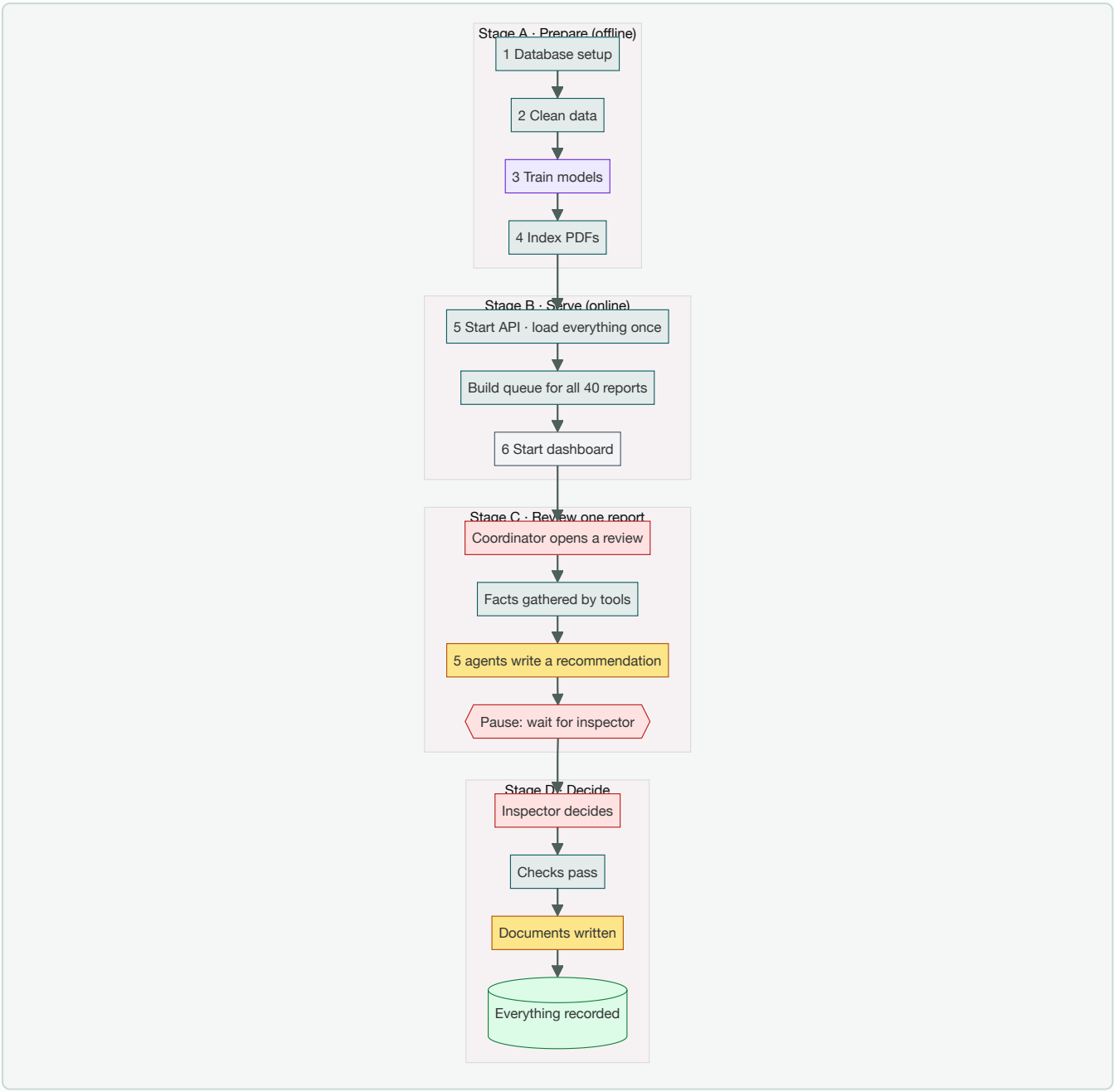
- Writing a requirement limit (40 mm, 1500 mm …) anywhere in code or config.
- Reading a requirement from any standard other than the report's own (or its linked type's).

PART 2

End-to-end workflow

Every stage in order: what goes in, what comes out, the rules, and how to build it well.

7 The whole journey



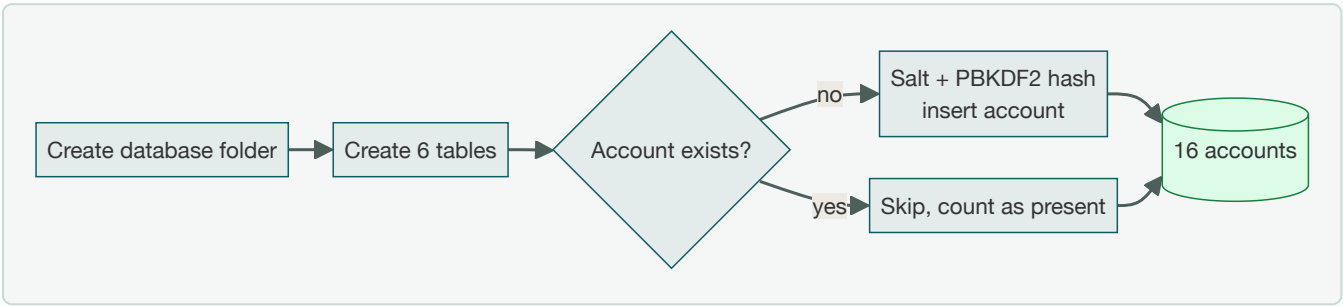
Step	Command	Creates	Expected result
1	`python3 run_database_setup.py`	streetcare.db, 6 tables	accounts created: 16 (second run: 0, already present 16)
2	`python3 run_data_pipeline.py`	processed/*.csv, quality report	e.g. history train 32738 → 30569
3	`python3 run_model_training.py`	3 models, 2 schemas, metrics	every model reaches its minimum
4	`python3 run_knowledge_indexing.py`	chroma_store	16 documents, 136 sections
5	`python3 run_api_service.py`	running API, checkpoint.db	port 8081
6	`python3 -m streamlit run run_dashboard.py`	running dashboard	port 8080

8 Database setup

INPUT
`SEED_USER_ACCOUNTS` in config

OUTPUT
`database/streetcare.db` with 6 tables and 16 accounts

MODULES
persistence/*, access/password_security



Role	Accounts	Notes
Highways Inspector	8 (2 per district)	can decide reports in their own district only
Street Care Coordinator	from seed list	opens reviews
Customer Services Adviser	from seed list	chat only

BEST PRACTICE

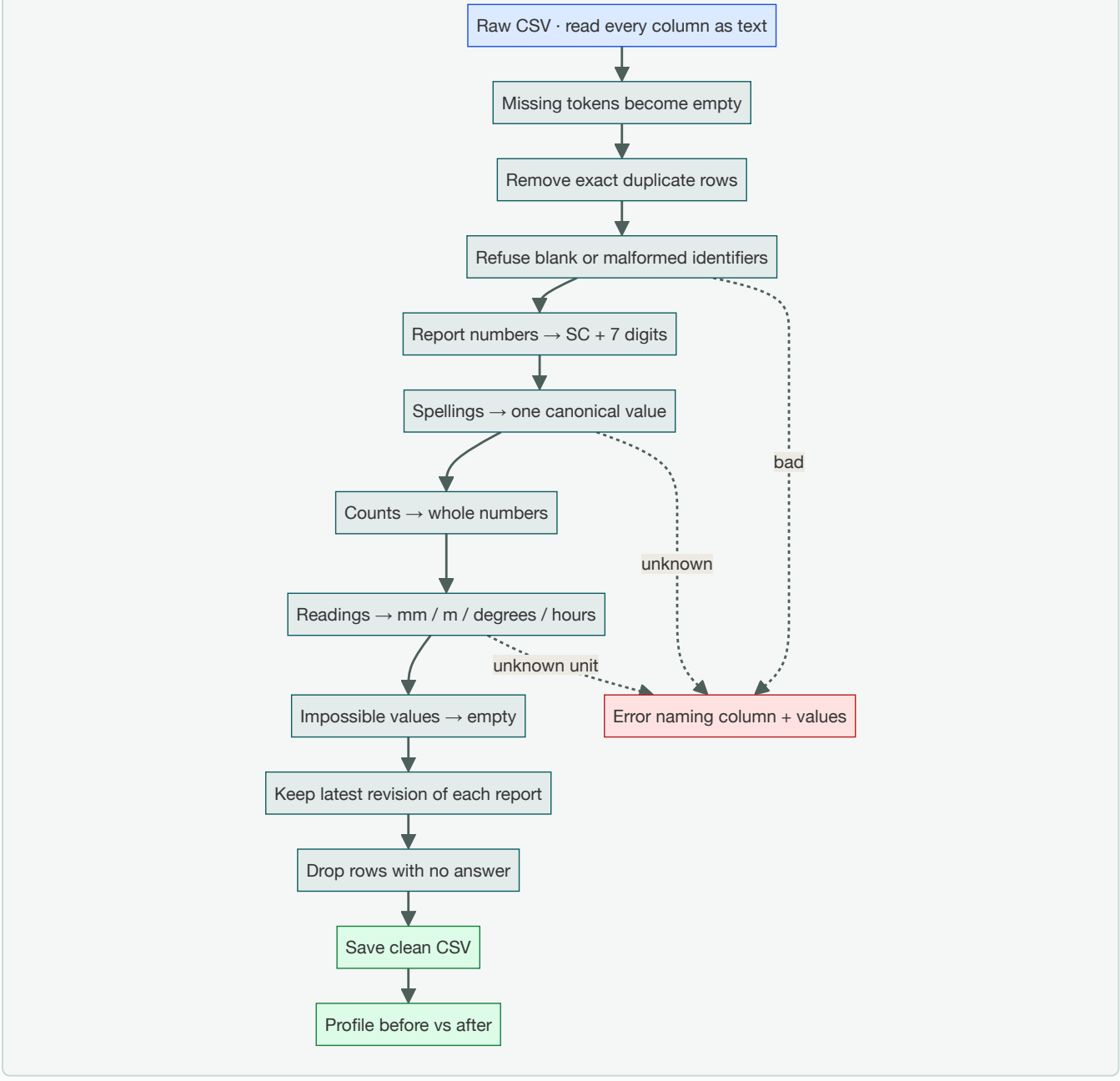
- **Idempotent:** running setup twice creates nothing new and reports "already present: 16".
- Passwords: random 16-byte salt, PBKDF2-HMAC-SHA256, 200,000 rounds. Never store or log the plain password.
- One engine and session factory in `persistence/database.py`; every write goes through `repository.py`.

9 Data pipeline

INPUT
5 raw training CSVs (and the 3 live CSVs, cleaned on demand)

OUTPUT
`processed/*_clean.csv` + `output/data_quality_report.json`

MODULES
data_pipeline/*



Conversions (worked exactly, rounded half up once)

As written	Read as	As written	Read as
SC0050118 / SC-50118 / 50118	SC0050118	4.5 cm	45 mm
1.75 in	44 mm	4 ft	1.2 m
45 min	0.8 hours	CW / Pothole or road surface	Carriageway
Y / true / yes	Yes	N/A / - / caller unsure	(missing)

Expected results

Dataset	Received	Kept	Removed	Answer column
report_history_train	32,738	30,569	2,169	reported_again
report_history_eval	8,202	7,648	554	reported_again
repair_logs_train	26,095	25,537	558	actual_crew_hours
repair_logs_eval	6,476	6,337	139	actual_crew_hours
statement_sentences	36,661	35,717	944	sentence_type
open_defect_reports (live)	45	40	5	none
street_works_register (live)	29	28 (23 live)	1	none

BEST PRACTICE

- Every cleaning function returns a **new** table and never changes the one it was given (`df.copy()`).
- Use `Decimal(str(value))`, multiply, then `quantize(step, ROUND_HALF_UP)`. Never round twice, never use float maths for conversions.
- Reset the index (0, 1, 2 …) after removing rows.
- Keep the latest revision wherever its rows sit in the file. Refuse two rows with the same report and revision.
- Fill gaps **inside the saved model pipeline**, not in the clean CSV. Real gaps (a water depth on a footway report) stay empty.
- The removal reasons in the quality report must add up to `rows_removed`. Write JSON with indent 2 and sorted keys so reruns are identical.

AVOID

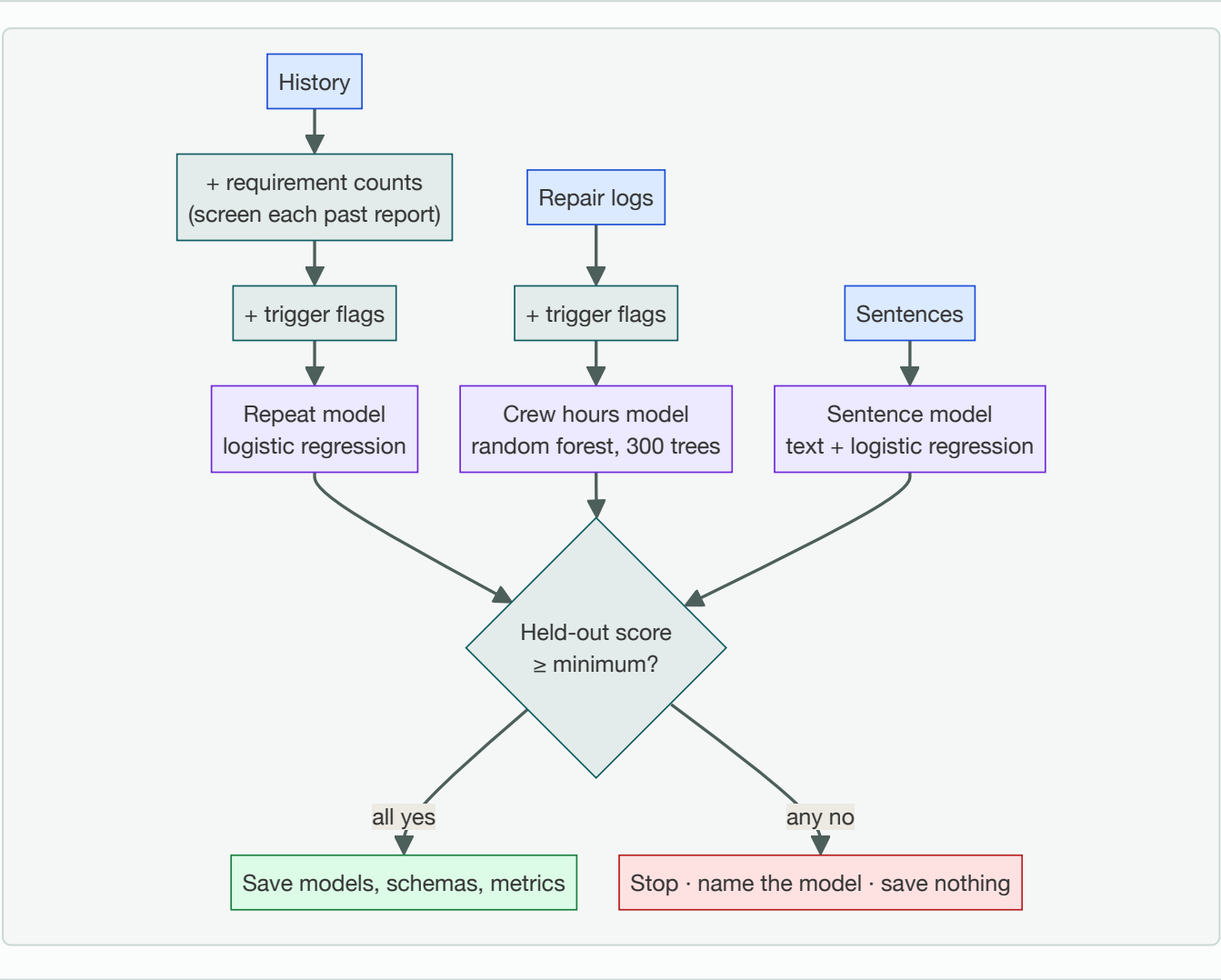
- Letting pandas guess types when reading raw files (always read as text).
- Silently dropping or "fixing" a value you do not recognise.

10 Model training

INPUT
Clean train/eval CSVs + the 5 defect standards

OUTPUT
`output/models/*.pkl`, feature schemas, `model_metrics.json`

MODULES
modeling/*, data_pipeline/feature_builder



Model	Predicts	Algorithm	Score	Minimum
Repeat	Will the defect be reported again? (Yes=1)	LogisticRegression, 3000 iterations	ROC AUC	0.80
Crew hours	Hours the repair really takes	RandomForest: 300 trees, min leaf 10, max features 0.7	R^2, MAE	0.86
Sentence	Which of 9 types a sentence is	Text features + LogisticRegression	macro F1	0.95

BEST PRACTICE

- One **pipeline** per model: step `preprocessing` (median fill + scaling for numbers, most-frequent fill + one-hot for categories) then step `estimator`. Save the whole pipeline.
- Fit only on train files; score on eval files. Report train, held-out and the gap to show over-fitting.
- Seed every random choice with `RANDOM_SEED = 1313`, so runs are repeatable.
- Save the feature order as JSON and use it to lay out live reports, so inputs always match.
- Use `handle_unknown="ignore"` in the encoder, so a new category at scoring time does not crash.
- Check every minimum **before** saving anything.

AVOID

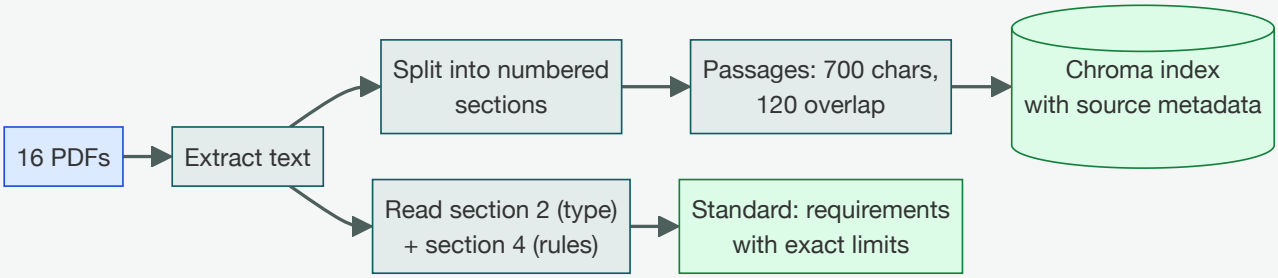
- Fitting the scaler or encoder on all data (leaks eval into training).
- Training on earlier revisions of a report.

11 Knowledge indexing

INPUT
16 PDFs in `data/docs`

OUTPUT
Chroma index (16 documents, 136 sections) + each standard's rules

MODULES
knowledge/*



Documents	Used for
5 defect standards	requirements (§4), what an inspection records (§6), works order (§7), resident update (§8)
8 statement guides	what the reviewer checks (§3) for each sentence type
Highways Policy	coordination triggers (§6), the three decisions (§7), documents and the brief (§8)
Report Rules	how every answer is read (units, rounding, worked example §13)
Past decisions casebook	indexed, but **excluded from chat**

Example rule read from the Carriageway standard: "Depth. The defect has a depth … of **at least 40** millimetres" becomes `{"kind": "defect_depth", "rules": {"minimum": Decimal("40")}}`.

BEST PRACTICE

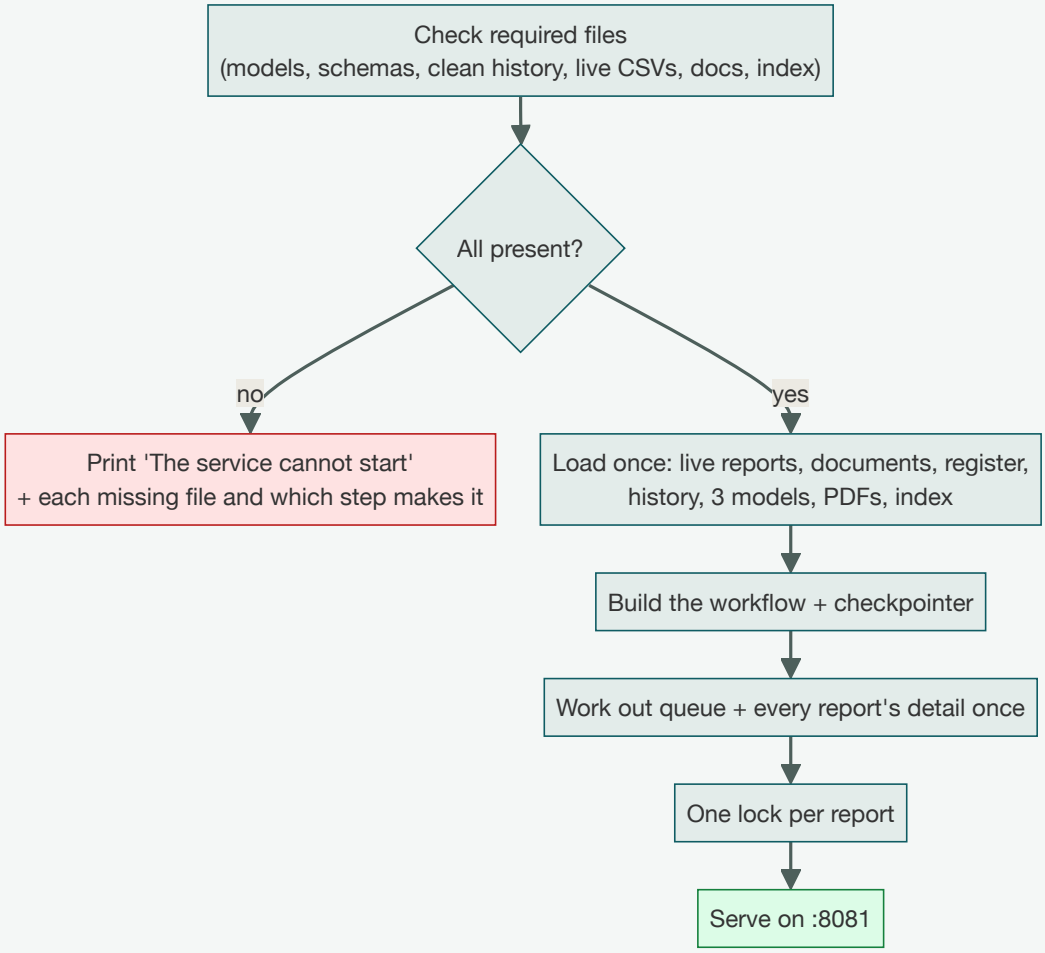
- Store document name, section number and title on every passage, so every statement can be traced to its source.
- Read a named section by **metadata filter**, not by similarity search. Similarity is for chat only.
- Check the type sentence in §2 matches the expected defect type; refuse a standard read for the wrong type.
- Rebuild the index from scratch on each run, so reruns give the same index.
- The run's self-check prints §6 of each standard. If two types show the same text, the index is wrong.

12 Service start-up

INPUT
Every file the build created

OUTPUT
A running API with everything held in memory

MODULES
run_api_service, agentic/nodes, api/routes



BEST PRACTICE

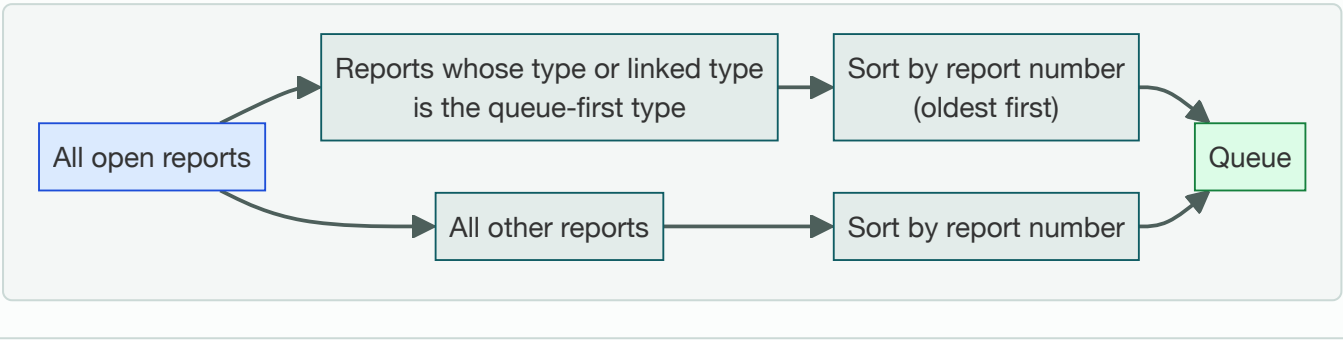
- **Fail fast:** check files before importing heavy modules, and say exactly what is missing and how to make it.
- Load models and the index **once** at import, never per request.
- Importing `run_api_service` must start nothing; only running it does (`if __name__ == "__main__"`).

13 Report queue

INPUT
40 open reports + review statuses

OUTPUT
Ordered list of queue rows

ROUTE
`GET /queue` (permission: queue)



Each row shows: position, report, type, street, district, channel, status, document count, requirement counts, permits, triggers, assessment, likelihood and band, expected hours, overrun, and disagreements.

Status: `AWAITING_REVIEW` → `AWAITING_DECISION` → `REPAIR_ORDERED` / `INSPECTION_REQUESTED` / `REFERRED_FOR_COORDINATION`.

AVOID

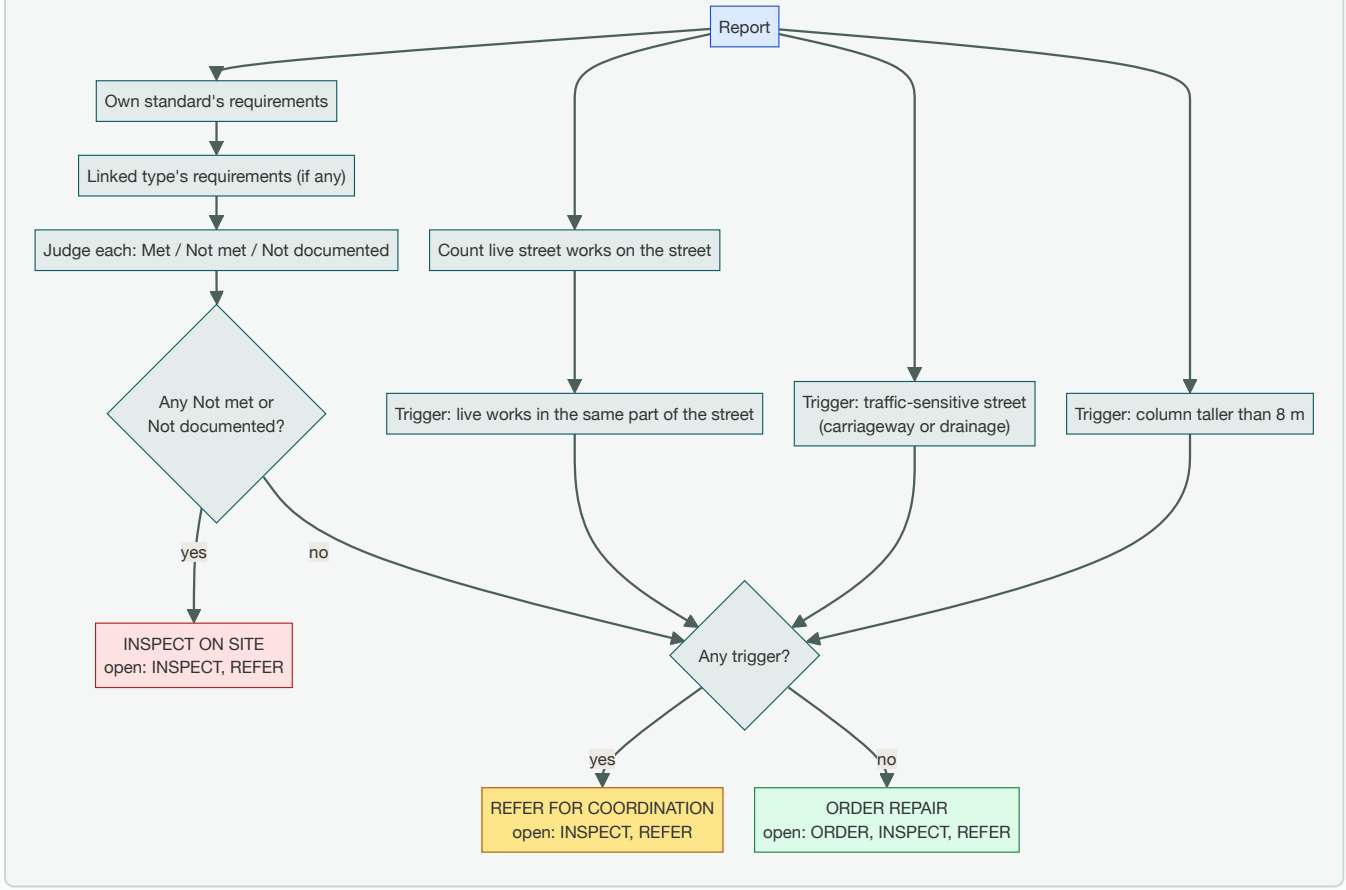
- Ordering the queue by a model's figure. The queue order is fixed by type group and report number only.

14 Report screening

INPUT
One report, its standards, the street works register

OUTPUT
Each requirement's result, triggers, assessment, decisions open

MODULE
tools/screening_tools



How each requirement is judged

Requirement kind	Met when
Yes/no (location fixed, photograph, highway land, column number)	Yes → Met · No → Not met · blank → Not documented
"at least" (depth, water depth)	value ≥ limit (exactly at the limit is Met)
"at least … at most" (width)	minimum ≤ value ≤ maximum
"less than"	value < limit (exactly at the limit is Not met)
Water reach	water on footway → Met · on carriageway → Met at or beyond the limit
Clear height	footway limit over the footway, carriageway limit over the carriageway
Column lean	lamp out or day burning → Met · leaning column → held to the limit

A "Not documented" result names only the blank answers it needs (a blank water location, overhang location or lighting fault is named alone).

WORKED EXAMPLE (Report Rules §13)

Contact-centre report `CW` linked `DR` on a `TS` street. Depth 1.25 in → 32 mm (Met); width 3.9 in → 99 mm (Not met); water 1.5 in → 38 mm (Not met); water on road, reach "caller unsure" (Not documented). Result: Met 1, Not met 2, Not documented 1 → **INSPECT ON SITE**. The traffic trigger also applies, but requirements come first.

BEST PRACTICE

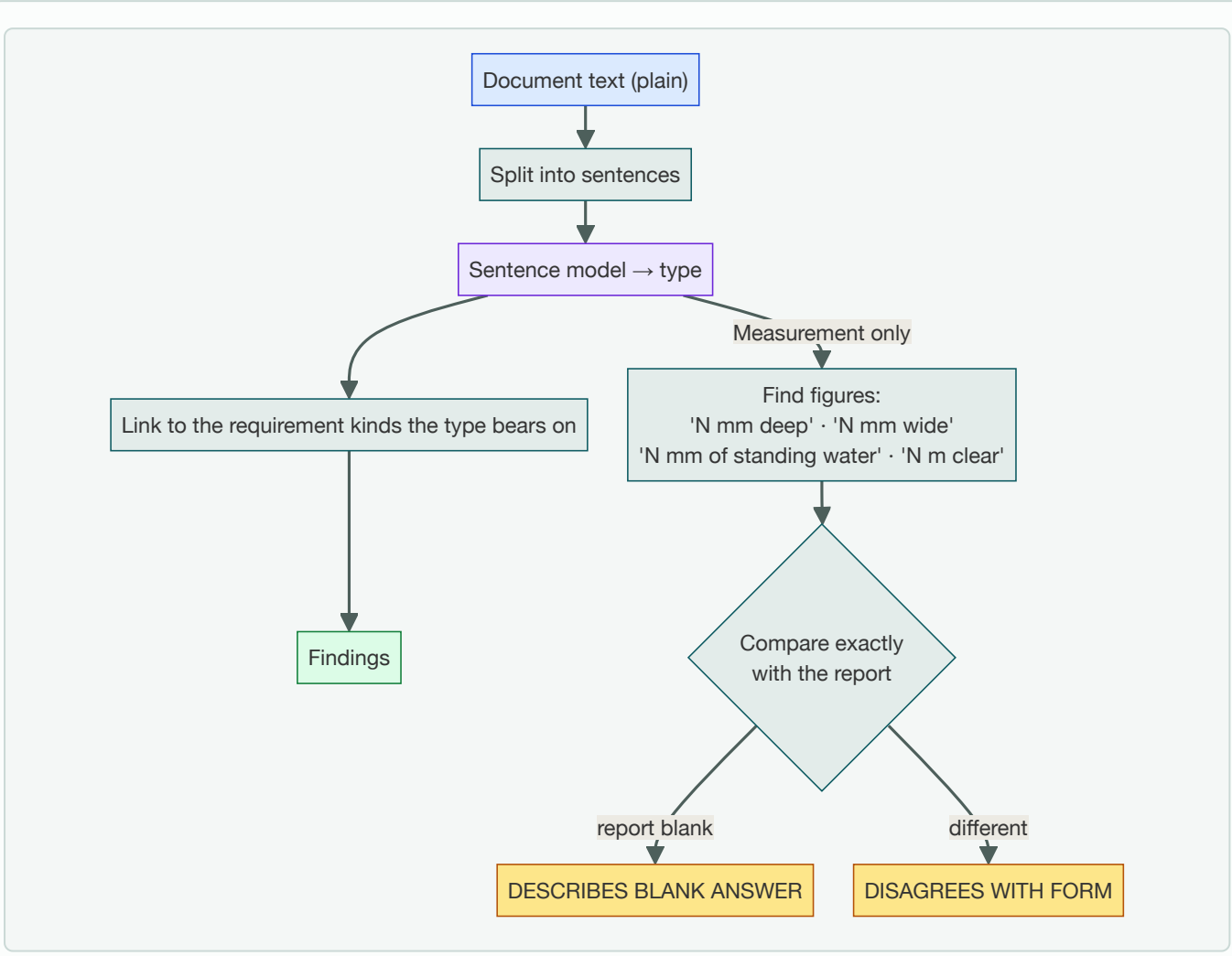
- Screening is a pure function: same report + same standards → same result. Easy to unit test.
- A blank answer never lets a report slip past coordination: a blank street category counts as traffic-sensitive and a blank column height counts as taller than 8 m.
- Only **live** permits on the **same street** count; vegetation is in neither part of the street.
- Each requirement keeps the name of the standard it came from.
- Write a test for every row of the table above, including the exact-limit cases.

15 Reading attached documents

INPUT
The report's attached documents

OUTPUT
Findings per sentence + disagreement flags

MODULE
tools/evidence_tools



Sentence type	Bears on
Measurement	depth, width, water depth, water reach, clear height, column lean
Location	location fixed, column number, highway land
Photograph	photograph
Weather and water	water depth, water reach
Hazard, Incident, Traffic and use, Previous reports, Background	none

BEST PRACTICE

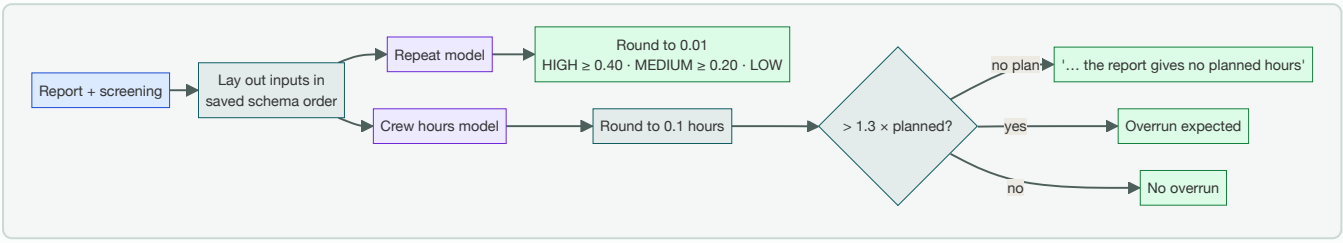
- A flag **never** changes the report's answer; it only alerts the inspector.
- Compare the figure unrounded, as an exact Decimal, with the report's own reading.

16 Predictions

INPUT
Report + screening (counts, trigger flags)

OUTPUT
Repeat likelihood + band; expected crew hours + overrun text

MODULE
tools/prediction_tools



BEST PRACTICE

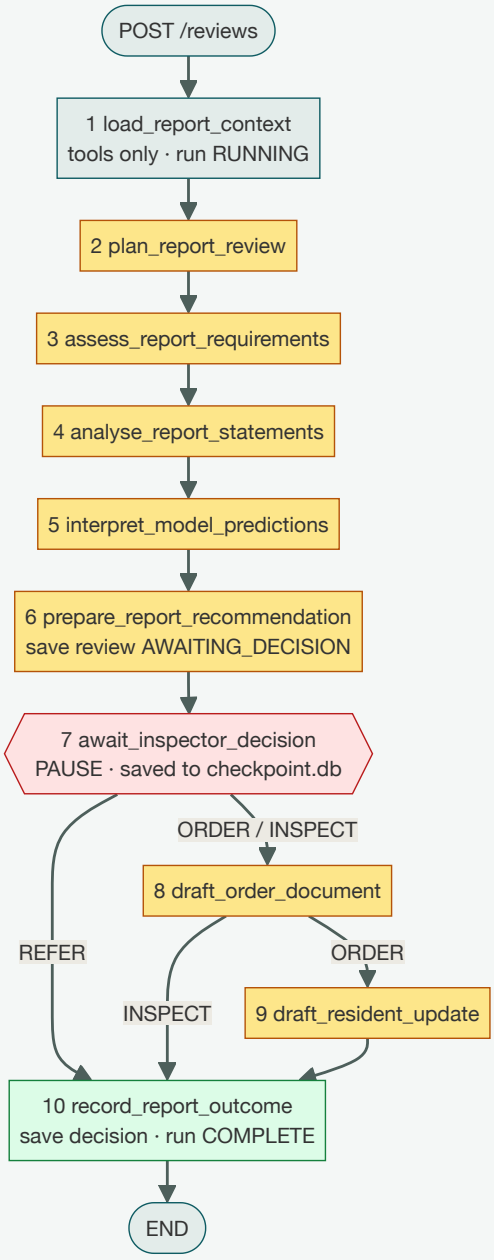
- Predictions inform the inspector only. They never order the queue, never change the assessment, and never appear in a works order, inspection request or resident update.
- Customer services never sees them.
- Round half up with `Decimal`, at the configured step.

17 Review workflow

INPUT
Report number, coordinator's request, coordinator, thread id

OUTPUT
A stored review, then a decision and its documents

MODULES
agentic/state, nodes, graph



Decision	Path after pause	Total LLM calls	Final status
ORDER REPAIR	8 → 9 → 10	7	REPAIR_ORDERED
INSPECT ON SITE	8 → 10	6	INSPECTION_REQUESTED
REFER FOR COORDINATION	10	5	REFERRED_FOR_COORDINATION

Workflow state (`ReportReviewState`)

Keys	Set by
report_reference, coordinator_request, coordinator_id, thread_id	API, at start
run_id, review_id, report_profile, report_documents, register_count, screening, evidence_findings, disagreements, likelihood, crew_hours	node 1
review_plan, requirements_analysis, evidence_review, prediction_analysis	nodes 2–5
review_decision	node 7, on resume
order_document, document_details, resident_update	nodes 8–9
llm_call_count	**added to** by each agent node (all other keys are replaced)

BEST PRACTICE

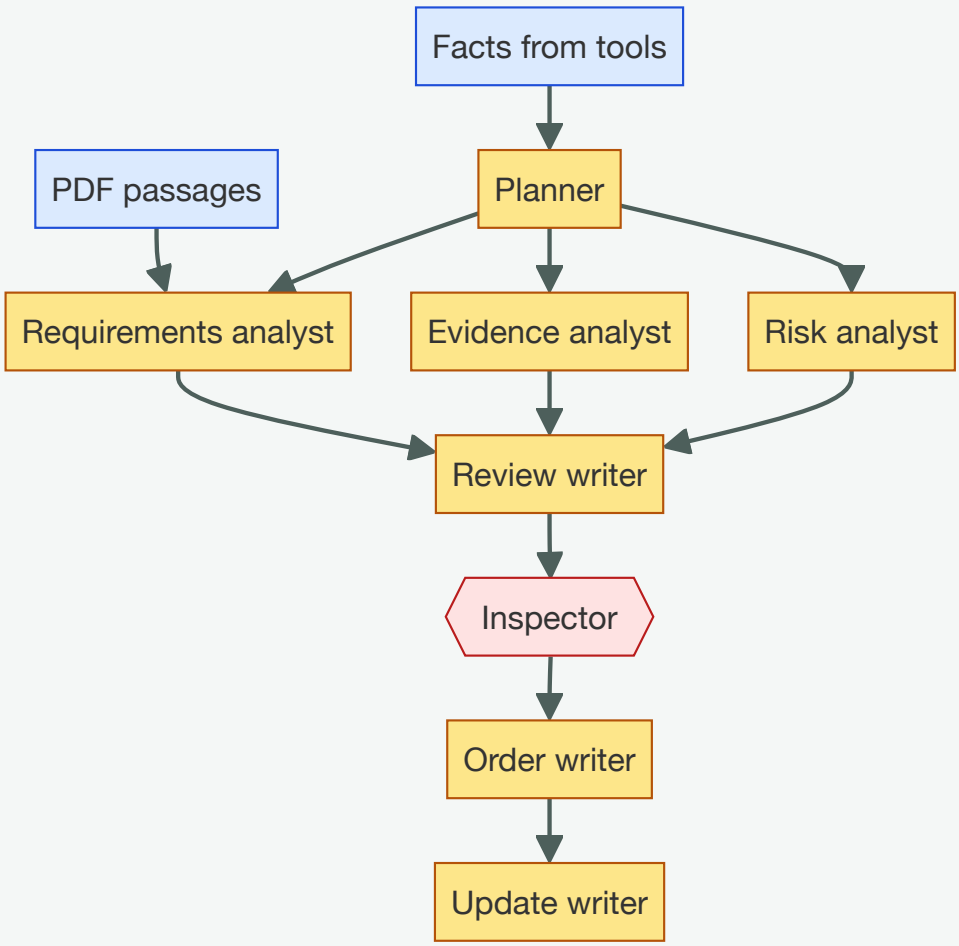
- Each node does one thing and returns **only** the keys it wrote.
- Declare `llm_call_count` with an add reducer (`Annotated[int, operator.add]`).
- Compile with a SQLite checkpointer whose connection allows use from more than one thread (`check_same_thread=False`).
- Node 7 writes nothing: a paused node runs again from its start when the run resumes.
- Node 6 and node 10 each write in **one transaction** (call + review + run), so the database is never half-updated.
- Exactly two conditional branches, both chosen by `route_report_decision`; every other connection is a plain edge.
- Building the workflow runs nothing and calls no model.

18 AI agents

INPUT
Facts from tools + passages from PDFs

OUTPUT
Labelled text fields

MODULES
agentic/agents, language_model



Agent	Returns
review_planner_agent	PRIORITY, CHECKS, RATIONALE
requirements_analyst_agent	REQUIREMENTS MET, REQUIREMENTS NOT MET, NOT DOCUMENTED, COORDINATION TRIGGERS, REQUIREMENTS SUMMARY
evidence_analyst_agent	EVIDENCE FOUND, DISAGREEMENTS, CHECKS TO MAKE, EVIDENCE SUMMARY
risk_analyst_agent	LIKELIHOOD, EXPECTED CREW HOURS, PREDICTION SUMMARY
review_writer_agent	RECOMMENDATION, RECOMMENDED DECISION (must be one still open)
order_writer_agent	SUBJECT, DOCUMENT
resident_update_writer_agent	HEADLINE, UPDATE

BEST PRACTICE

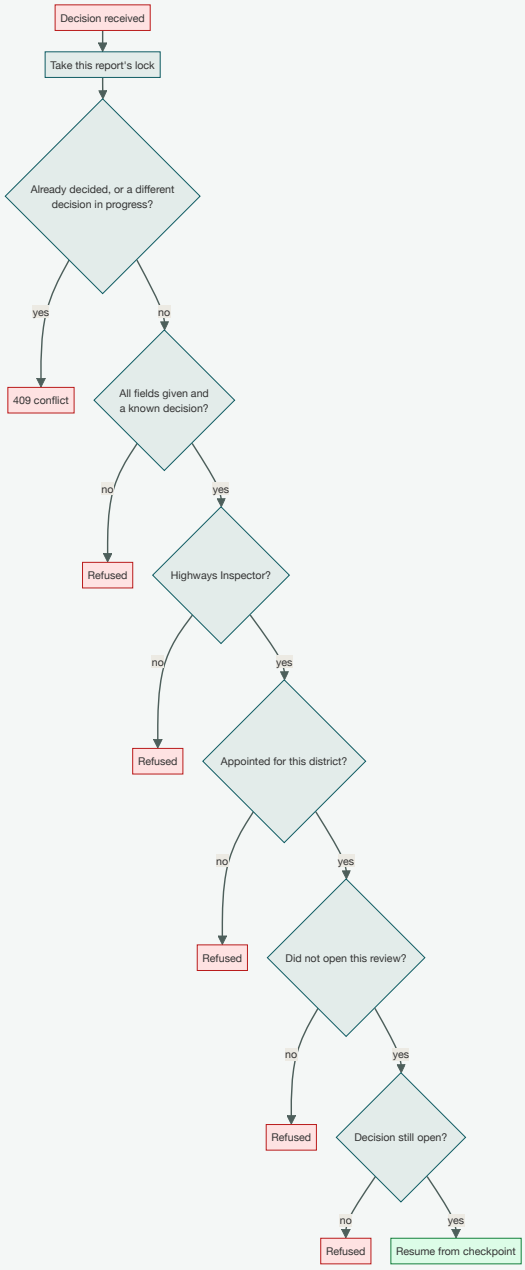
- Each agent is **exactly one** LLM call. The prompt tells it to use only what it is given.
- Ask for fixed labelled fields; parse them strictly and refuse a reply with missing, extra or multi-line fields.
- Refuse an empty reply, a reply that did not finish normally, or one with no token counts.
- Read the API key and model name at call time; refuse clearly if either is missing.
- Record every accepted call: agent, model, prompt/response/total tokens, latency in ms.
- Outgoing documents: signed "Street Care Office, Wrenford Borough Council"; no "likelihood", "probability" or "expected crew hours"; no `[ ]` template slots; no staff names in resident updates.

19 Human decision gate

INPUT
`POST /reviews/{id}/decision` with decision + reason

OUTPUT
Accepted (run resumes) or 409 with the reason

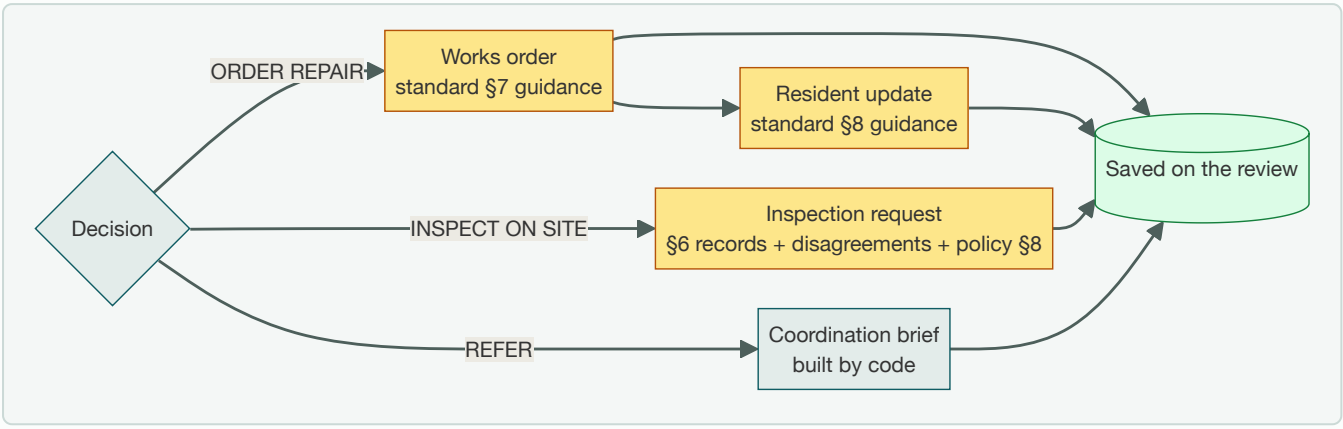
MODULE
tools/decision_tools, api/routes



BEST PRACTICE

- All checks run **before** the run resumes, so a refused decision costs no LLM call.
- One lock per report stops two decisions racing. The same decision sent again after a failure finishes the run.
- Resume through the checkpoint with the same `thread_id`; it works after a restart and from another process.

20 Decision documents

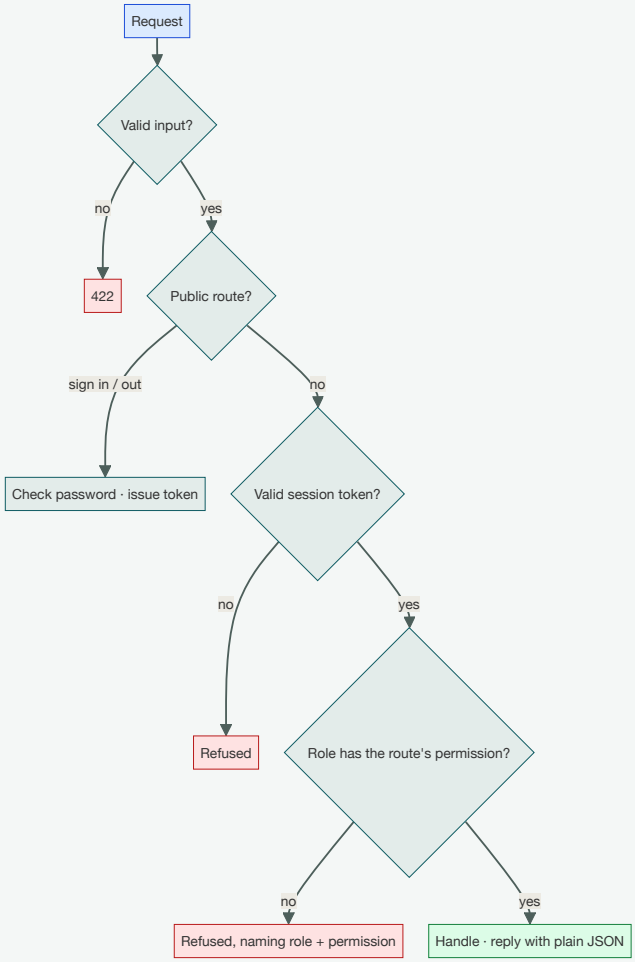


Document	Contains	Never contains
Works order	defect types, street, location, measurements, guidance for each type screened	model figures
Resident update	what will happen, from the works order details	staff names, model figures
Inspection request	what the inspector must record, the disagreements to settle	model figures
Coordination brief	defect types, street, permits and triggers	(written by code, no LLM)

BEST PRACTICE

- Build the document **details** in code (tools), and let the agent only write the prose around them.
- For a linked report, include guidance from both standards, each as "Under the <type> standard: …".

21 API and access control

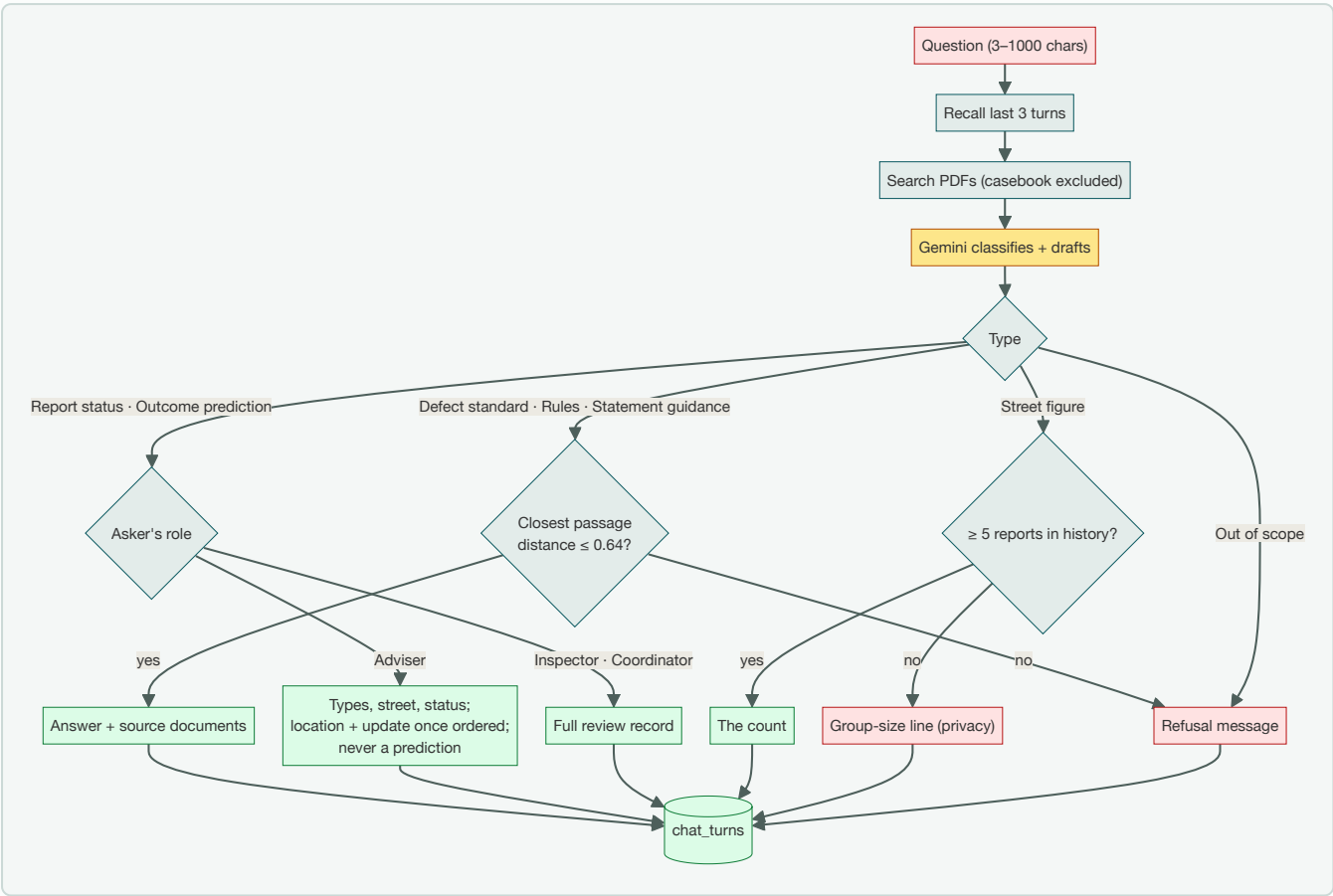


Route	Permission	Inspector	Coordinator	Adviser
POST /auth/signin, /auth/signout	public	✔	✔	✔
GET /queue, /queue/{ref}	queue	✔	✔	–
POST /reviews (201)	review	✔	✔	–
POST /reviews/{id}/decision	decision	✔	–	–
GET /reviews, /reviews/{id}/calls, /audit/chat-turns	audit	✔	✔	–
POST /chat/ask, GET /chat/turns	chat	✔	✔	✔

BEST PRACTICE

- Put every protected route on a router guarded by `authorize_request`, so no route can forget the check.
- Session token in the `X-Session-Token` header; 24 random bytes; a new sign-in ends the old session.
- Pydantic validates input before any work; replies hold only plain Python values (no pandas/NumPy types).
- One failure handler turns errors into consistent JSON replies.
- Only the 3 POST routes for reviews, decisions and chat may spend LLM calls.

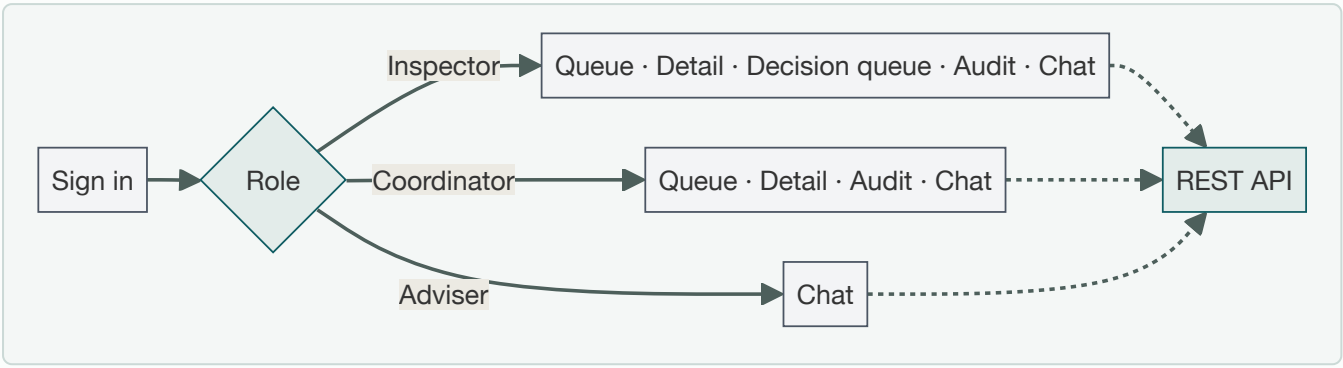
22 Chat assistant



BEST PRACTICE

- The AI only classifies and answers document questions. **Code** writes every answer about a report or a street.
- Street counts come from the report **history**, never the live queue, and are counted by defect type only.
- Store every question, answer, type, sources, distance and whether it was refused.

23 Dashboard



Page	Shows / does	Calls
Report queue	all open reports, filter by type	GET /queue
Report detail	screening, evidence, predictions; open a review	GET /queue/{ref}, POST /reviews
Decision queue	reviews waiting; decide	GET /reviews, POST /reviews/{id}/decision
Audit trail	reviews, LLM calls with tokens and time, chat turns	GET /reviews, /reviews/{id}/calls, /audit/chat-turns
Street care chat	ask questions; own history	POST /chat/ask, GET /chat/turns

BEST PRACTICE

- All HTTP goes through `api_client.call_service` with a timeout (300 s), a spinner message and a clear failure message.
- No business rule in the dashboard. If a number is shown, it came from the API.
- Hiding a page is for convenience only. Security is enforced by the API.

24 Storage and audit

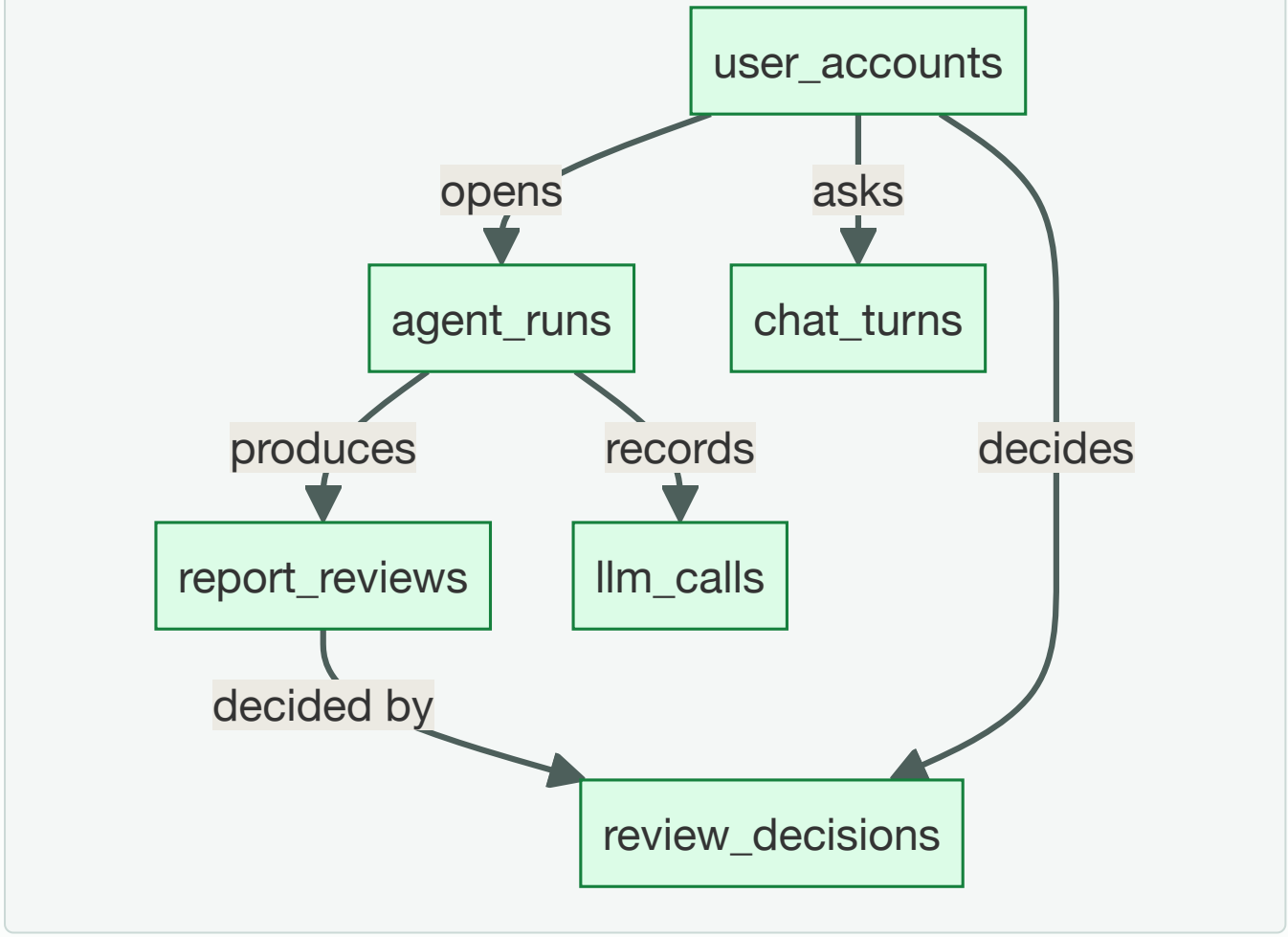


Table	Key columns
user_accounts	username, display_name, role, district, password_hash, password_salt, session_token
agent_runs	run_id, thread_id, report_reference, coordinator_id, coordinator_request, status, llm_call_count
report_reviews	review_id, run_id, report data, requirements and counts, permits, triggers, assessment, decisions_open, evidence, disagreements, likelihood, crew hours, 5 agent texts, recommendation, status, outcome documents (37 columns)
review_decisions	decision_id, review_id, run_id, decision, decided_by, role, district, decision_reason
llm_calls	call_id, run_id, agent_name, model_name, prompt/response/total tokens, latency_milliseconds
chat_turns	turn_id, username, user_role, question, type, report, street, defect type, answer, sources, closest_distance, was_refused

Run status: `RUNNING` → `AWAITING_DECISION` → `COMPLETE` (or `FAILED`).

BEST PRACTICE

- Table and column names exactly as specified, in lower case.
- Group related writes in one transaction; roll back on any error.
- Identifiers: `RUN-` / `REV-` + 12 random upper-case hex characters.

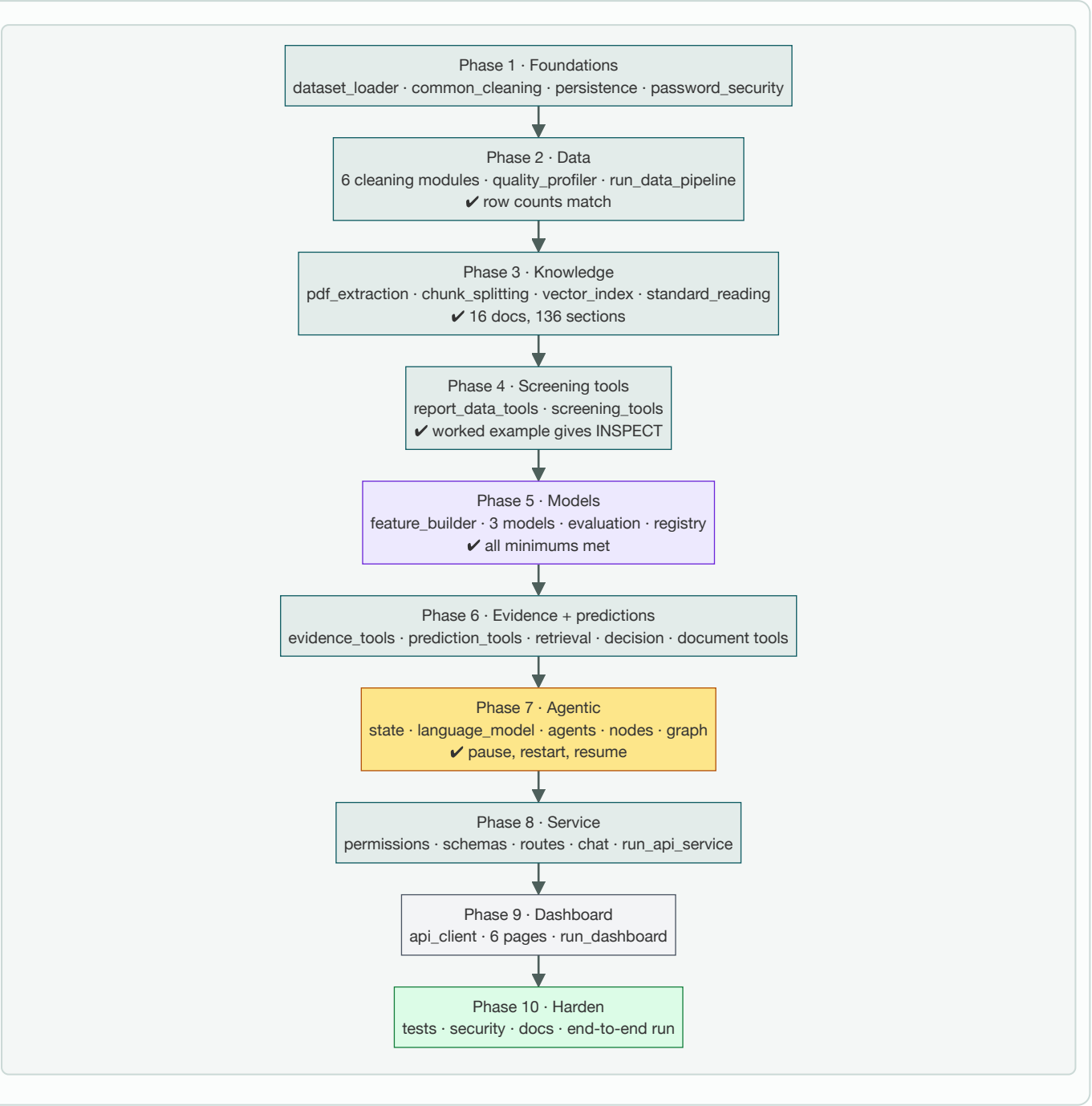
PART 3

Engineering practice

How to build, test, secure, run and finish the project.

25 Build order

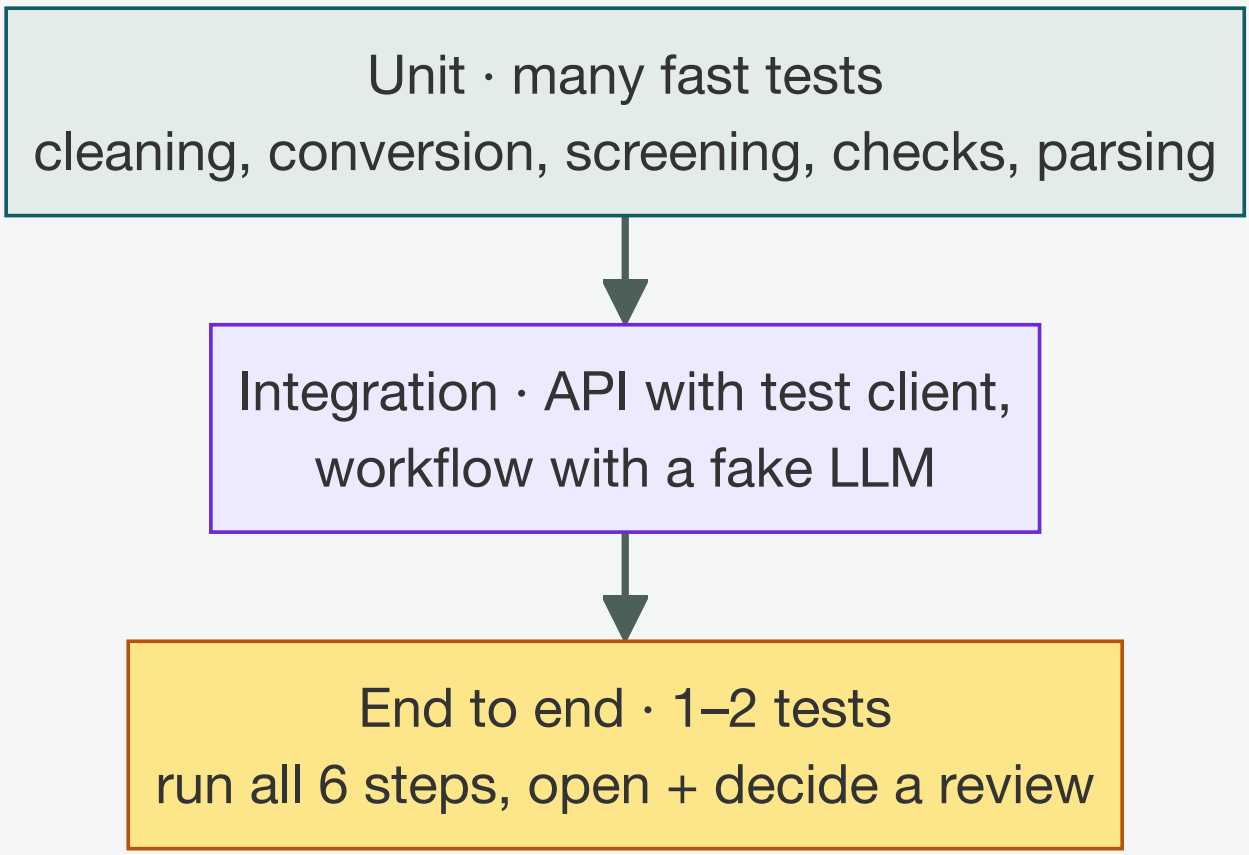
Build bottom-up. Each phase can be tested before the next starts.



WHY THIS ORDER

- Screening (Phase 4) must exist before training, because the repeat model learns from requirement counts the screening produces.
- Agents come late: they only interpret facts, so the facts must be right first.

26 Testing strategy



Area	Must-have tests
Loading	every NULL_TOKEN becomes empty in any case; nothing else does; whitespace stripped
Conversion	4.5 cm → 45 · 1.75 in → 44 · 4 ft → 1.2 · 45 min → 0.8 · unknown unit raises
References	SC0050118, SC-50118, 50118 → SC0050118; bad form raises
Revisions	latest kept wherever it sits; duplicate (report, revision) raises
Pipeline	exact kept counts for all 5 datasets; removal reasons add up
Standards	each type reads its own figures; carriageway ≠ footway depth
Screening	each judging rule, exact-limit cases, worked example → INSPECT; blank category counts as traffic-sensitive
Evidence	2.1 m vs 2.4 flagged; blank answer flagged; flags never change answers
Models	minimums met; save/load round-trip scores the same
Agents	parser refuses missing/extra/multi-line fields; recommended decision must be open
Workflow	5 calls before pause; 7/6/5 total; pause survives rebuilding the graph
Gate	wrong role, wrong district, own review, closed decision, already decided all refused
Access	each role × each route; adviser cannot reach queue; no token refused
Chat	adviser never gets a prediction; street figure hidden under 5; out of scope refused

BEST PRACTICE

- Replace Gemini with a **fake** in tests that returns fixed labelled replies: fast, free and repeatable.
- Use a temporary database and checkpoint file per test.
- Name tests after the rule they prove, e.g. `test_width_at_maximum_is_met`.

27 Error handling

Situation	Behaviour
Unknown value, unit or identifier in data	`ValueError` naming the column and the values; run stops
Blank required column / repeated reference	`ValueError` naming the column and the count or values
Model below minimum	stop, name the model, save nothing
Required file missing at start-up	"The service cannot start", list each missing file and the step that makes it
Invalid API input	422 before any work
Not signed in / not permitted	refused with the configured message
Decision conflict or refusal	409 with the reason; no LLM call spent
Bad LLM reply	refused, naming the agent; the call is not accepted into the review
Missing Gemini key or model	refused at call time, naming both settings
Failure inside a run	run marked FAILED; the same decision can be sent again to finish it

BEST PRACTICE

- Use the exact message templates from config; tests and the marking suite read them.
- Fail loudly and early. A wrong answer that looks right is worse than a clear error.

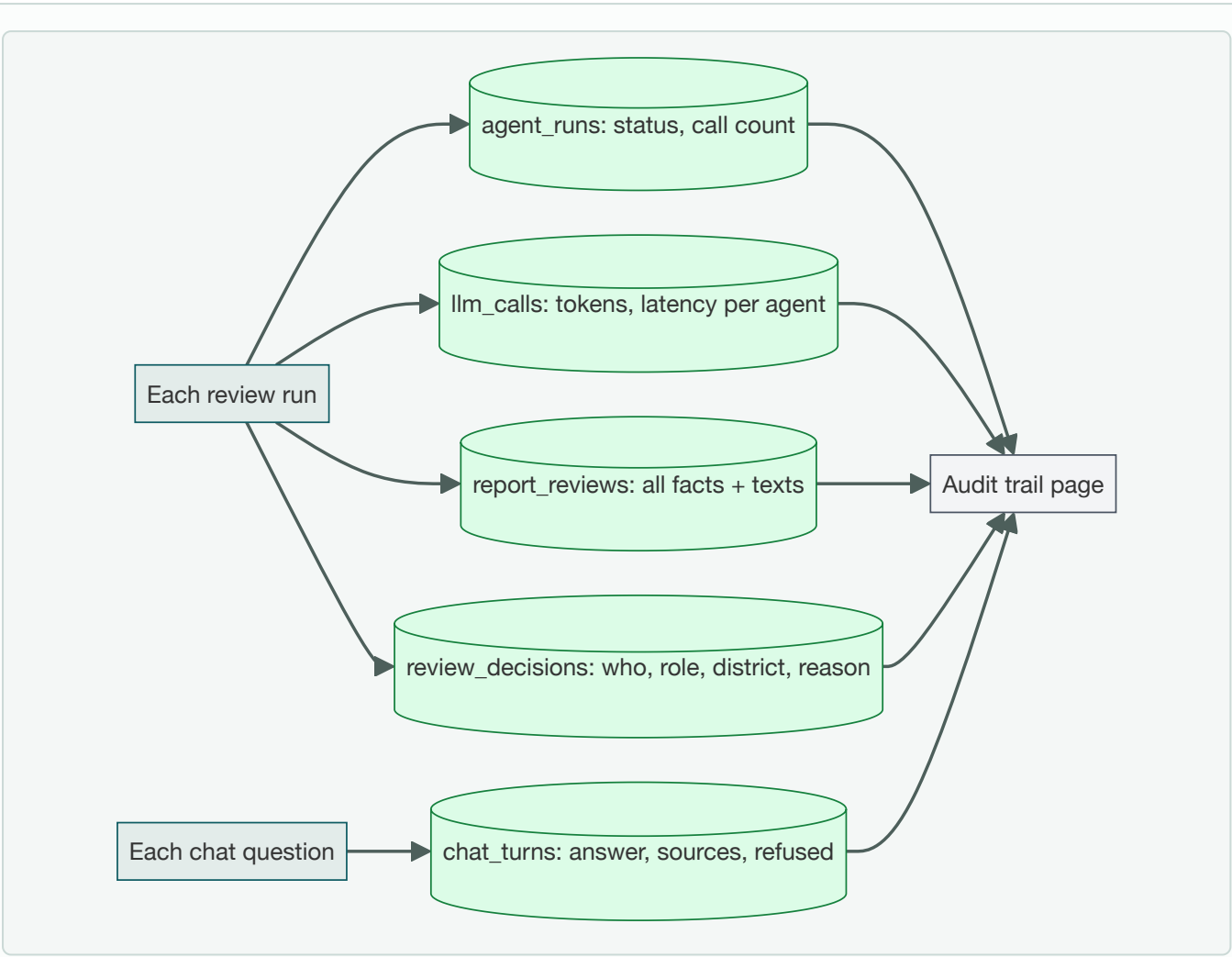
28 Security

FIX NOW: SECRETS IN THE REPOSITORY

- `Project/.env` contains a real `GEMINI_API_KEY` and is committed to git. Rotate the key, remove the file from git, and add `.env` to `.gitignore`.
- `.github/workflows/build.yml` has a `SONAR_TOKEN` written into the file. Rotate it and move it to repository secrets (`${{ secrets.SONAR_TOKEN }}`).

Area	Practice
Passwords	PBKDF2-HMAC-SHA256, 200,000 rounds, 16-byte random salt per account; compare with `hmac.compare_digest`
Sessions	`secrets.token_hex(24)`; one active session per account; cleared on sign-out
Authorisation	checked by the API on every request; district checks for decisions; separation of duties (opener ≠ decider)
Input	Pydantic limits on every field (question 3–1000 characters, decision from a fixed list)
Privacy	no predictions to customer services; no street figure under 5 reports; no staff names in resident updates
LLM safety	agents get facts, not database access; replies are validated before use; chat answers only from council documents
Logging	never log passwords, tokens or the API key
SQL	always through SQLAlchemy; never build SQL from strings

29 Observability



BEST PRACTICE

- `GET /reviews/{id}/calls` shows call count against the expected count (5, 6 or 7), total tokens and total latency. A mismatch means something went wrong.
- Pipeline and training write their evidence to `data_quality_report.json` and `model_metrics.json`; keep them with each release.
- Print short, fixed-format console lines (the marking suite reads them).

30 Performance

Practice	Why
Load models, PDFs, index and live files once at start-up	requests stay fast; no repeated disk reads
Work out the queue and every report's detail once	`GET /queue` is instant and makes no model calls
Embedding model on CPU, small (`all-MiniLM-L6-v2`)	runs anywhere, no GPU needed
One LLM call per agent, fixed fields, temperature 0.2	predictable cost and latency; easier to validate
Compressed model files (joblib level 3)	smaller files, fast enough to load
Dashboard timeout 300 s with a waiting message	a review makes 5 LLM calls; users see progress
One lock per report, not a global lock	different reports can be decided at the same time

31 Running and operating

```
cd Project
pip install -r requirements.txt
# put GEMINI_API_KEY and GEMINI_MODEL in .env (never commit it)

python3 run_database_setup.py        # once (safe to repeat)
python3 run_data_pipeline.py         # when raw data changes
python3 run_model_training.py        # after the pipeline
python3 run_knowledge_indexing.py    # when PDFs change
python3 run_api_service.py           # terminal 2, keep running
python3 -m streamlit run run_dashboard.py   # terminal 3
```

Situation	What to do
API says "cannot start"	run the step named next to each missing file
Machine restarted mid-review	start the API again; paused reviews resume from `checkpoint.db` when decided
A PDF standard changed	rerun indexing, then training (requirement counts change), then restart the API
New live reports	replace files in `data/input/` and restart the API
A decision failed half-way	send the same decision again; it finishes the run
Start again from clean	delete `processed/`, `output/`, `database/`; rerun steps 1–4

BEST PRACTICE

- Back up `database/streetcare.db` and `checkpoint.db` together; they must match.
- Keep generated folders (`processed/`, `output/`, `database/`) out of git.

32 Definition of done

Data and models

- □ Pipeline prints the exact received/kept/removed counts
- □ Quality report: removal reasons add up; same file on rerun
- □ All 3 models meet their minimums; metrics saved
- □ No requirement limit anywhere in code or config
- □ 16 documents, 136 sections indexed; §6 read back per type

Review workflow

- □ Worked example screens to INSPECT ON SITE
- □ 5 calls before the pause; 7 / 6 / 5 in total
- □ Pause survives an API restart
- □ All gate refusals work; opener cannot decide
- □ Documents carry no model figures; signed correctly

Service and UI

- □ Setup is idempotent (16 then 0 created)
- □ Every route refuses roles without permission
- □ API refuses to start if a model is missing
- □ Each role sees only its pages
- □ Every figure on screen comes from the API

Chat and quality

- □ Adviser never gets a prediction
- □ No street figure under 5 reports
- □ Every chat turn and LLM call is stored
- □ Tests pass with a fake LLM
- □ Secrets rotated and out of git
- □ Names match the specification exactly

33 Glossary

Term	Meaning
Defect standard	The PDF for one defect type, stating its requirements and limits
Linked defect type	A second type the same defect also is; the report must meet both standards
Requirement	One rule a report must meet (e.g. depth at least 40 mm)
Met / Not met / Not documented	The three results of checking a requirement
Coordination trigger	Live street works, traffic-sensitive street, or high column
Assessment	ORDER REPAIR, INSPECT ON SITE or REFER FOR COORDINATION, worked out by code
Decisions open	The decisions the inspector may still choose for an assessment
Revision	A re-sent report; only the highest revision counts
Street works permit	Permission for works on a street; live or cancelled
Carriageway / Footway	The road part for vehicles / the pavement
Repeat likelihood	Model's chance the defect is reported again after repair
Expected crew hours	Model's estimate of how long the repair takes
Overrun	Expected hours more than 1.3 × planned hours
Disagreement flag	A document's measurement differs from the report or fills a blank
Gate	The pause where the workflow waits for a Highways Inspector
Checkpoint	The saved state of a paused workflow (`checkpoint.db`)
Agent	One Gemini call that interprets given facts and returns labelled fields